

PRÁCTICA GUIADA – ETAPA 8

PROYECTO "SERVICIOS TÉCNICOS"

Módulo Servicios – Relaciones Cliente + Técnico + Servicio

Flask • MySQL • React • JavaScript • CSS • Thunder Client • Git/GitHub

1. Propósito

En esta etapa el proyecto deja de trabajar solamente con CRUD independientes. Se construirá el módulo Servicios y se establecerá la relación entre Cliente, Técnico y Servicio. El alumno revisará claves primarias y foráneas, utilizará JOIN en MySQL, creará una API REST para Servicios y desarrollará en React una pantalla protegida que permita seleccionar un cliente y un técnico existentes.

2. Objetivos

- Identificar las relaciones reales de la base tecnico.
- Revisar claves primarias y foráneas con DESCRIBE y SHOW CREATE TABLE.
- Comprender la relación Cliente–Técnico–Servicio.
- Crear endpoints GET para catálogos y Servicios.
- Crear POST, PUT y DELETE para Servicios.
- Utilizar JOIN para mostrar nombres en lugar de solo IDs.
- Consumir las APIs desde React.
- Crear el módulo protegido /servicios.
- Probar cada operación con Thunder Client.
- Sincronizar el proyecto con GitHub.

3. Requisitos previos

- Etapa 7 terminada.
- CRUD de Clientes y Técnicos funcionando.
- Login y RutaProtegida funcionando.
- Flask, React, MySQL y Thunder Client disponibles.
- Base de datos tecnico funcionando.

4. Analizar el modelo de datos

Antes de programar, identifica las columnas y relaciones reales. No inventes campos.

```
USE tecnico;
SHOW TABLES;
DESCRIBE cliente;
DESCRIBE tecnico;
DESCRIBE servicio;
```



Después ejecuta:

```
SHOW CREATE TABLE cliente;
SHOW CREATE TABLE tecnico;
SHOW CREATE TABLE servicio;
```

Identifica la clave primaria de cada tabla, las claves foráneas de servicio, nombres de columnas, tipos y campos obligatorios.

Los ejemplos usan cliente, tecnico y servicio. Si tu base utiliza nombres diferentes, sustitúyelos en toda la práctica.

5. Relación conceptual

```
CLIENTE 1 ──────── N SERVICIO N ──────── 1 TÉCNICO
```

Un cliente puede tener varios servicios y un técnico puede atender varios servicios. Cada servicio se asocia con un cliente y un técnico mediante claves foráneas. La estructura exacta depende de tu base de datos.

6. Ejemplo de campos

Para explicar los ejemplos se supone que servicio contiene:

```
id
cliente_id
tecnico_id
descripcion
estado
fecha
```

Adapta los nombres y campos a tu esquema real.

7. Iniciar servidores

```
cd backend
venv\Scripts\activate
python -m app.main

cd frontend
npm run dev
```

API: http://127.0.0.1:5000 | Frontend: http://localhost:5173/



8. Crear servicios.py

backend/app/routes/servicios.py

```
from flask import Blueprint, jsonify, request
from config.conexion import conectar

servicios_bp = Blueprint("servicios", __name__)
```

9. GET – Servicios con JOIN

```
@servicios_bp.route("/api/servicios", methods=["GET"])
def obtener_servicios():
    conexion = conectar()
    if conexion is None:
        return jsonify({"success": False, "message": "Sin conexión a BD"}), 500
    cursor = None
    try:
        cursor = conexion.cursor(dictionary=True)
        consulta = """
        SELECT s.id, s.descripcion, s.estado, s.fecha,
               s.cliente_id, c.nombre AS cliente_nombre,
               s.tecnico_id, t.nombre AS tecnico_nombre
        FROM servicio s
        INNER JOIN cliente c ON s.cliente_id = c.id
        INNER JOIN tecnico t ON s.tecnico_id = t.id
        ORDER BY s.id DESC
        """
        cursor.execute(consulta)
        return jsonify({"success": True, "servicios": cursor.fetchall()}), 200
    except Exception as e:
        return jsonify({"success": False, "message": str(e)}), 500
    finally:
        if cursor: cursor.close()
        conexion.close()
```

Ajusta los JOIN a los nombres reales de tus tablas y columnas.

10. GET – Catálogo de Clientes

```
@clientes_bp.route("/api/clientes/catalogo", methods=["GET"])
def catalogo_clientes():
    conexion = conectar()
    if conexion is None:
        return jsonify({"success": False, "message": "Sin conexión a BD"}), 500
    cursor = None
    try:
        cursor = conexion.cursor(dictionary=True)
        cursor.execute("SELECT id, nombre FROM cliente ORDER BY nombre")
        return jsonify({"success": True, "clientes": cursor.fetchall()}), 200
    except Exception as e:
```




```
        return jsonify({"success": False, "message": str(e)}), 500
    finally:
        if cursor: cursor.close()
        conexion.close()
```

Agrega esta función al Blueprint existente de clientes.py; no dupliques el Blueprint.

11. GET – Catálogo de Técnicos

```
@tecnicos_bp.route("/api/tecnicos/catalogo", methods=["GET"])
def catalogo_tecnicos():
    conexion = conectar()
    if conexion is None:
        return jsonify({"success": False, "message": "Sin conexión a BD"}), 500
    cursor = None
    try:
        cursor = conexion.cursor(dictionary=True)
        cursor.execute("SELECT id, nombre FROM tecnico ORDER BY nombre")
        return jsonify({"success": True, "tecnicos": cursor.fetchall()}), 200
    except Exception as e:
        return jsonify({"success": False, "message": str(e)}), 500
    finally:
        if cursor: cursor.close()
        conexion.close()
```

12. POST – Crear Servicio

```
@servicios_bp.route("/api/servicios", methods=["POST"])
def crear_servicio():
    datos = request.get_json(silent=True)
    if not isinstance(datos, dict):
        return jsonify({"success": False, "message": "JSON válido requerido"}), 400

    cliente_id = datos.get("cliente_id")
    tecnico_id = datos.get("tecnico_id")
    descripcion = datos.get("descripcion")
    estado = datos.get("estado")
    fecha = datos.get("fecha")

    if not cliente_id or not tecnico_id or not descripcion:
        return jsonify({"success": False, "message": "Cliente, técnico y descripción son
obligatorios"}), 400

    conexion = conectar()
    if conexion is None:
        return jsonify({"success": False, "message": "Sin conexión a BD"}), 500
    cursor = None
    try:
        cursor = conexion.cursor()
        cursor.execute(
```




```
            """INSERT INTO servicio
            (cliente_id, tecnico_id, descripcion, estado, fecha)
            VALUES (%s, %s, %s, %s, %s)""",
            (cliente_id, tecnico_id, descripcion.strip(), estado, fecha)
        )
        conexion.commit()
        return jsonify({"success": True, "message": "Servicio creado", "id": cursor.lastrowid}), 201
    except Exception as e:
        conexion.rollback()
        return jsonify({"success": False, "message": str(e)}), 500
    finally:
        if cursor: cursor.close()
        conexion.close()
```

Incluye o elimina campos según las columnas obligatorias reales de servicio.

13. PUT – Actualizar Servicio

```
@servicios_bp.route("/api/servicios/<int:servicio_id>", methods=["PUT"])
def actualizar_servicio(servicio_id):
    datos = request.get_json(silent=True)
    if not isinstance(datos, dict):
        return jsonify({"success": False, "message": "JSON válido requerido"}), 400

    cliente_id = datos.get("cliente_id")
    tecnico_id = datos.get("tecnico_id")
    descripcion = datos.get("descripcion")
    estado = datos.get("estado")
    fecha = datos.get("fecha")

    if not cliente_id or not tecnico_id or not descripcion:
        return jsonify({"success": False, "message": "Campos obligatorios faltantes"}), 400

    conexion = conectar()
    if conexion is None:
        return jsonify({"success": False, "message": "Sin conexión a BD"}), 500
    cursor = None
    try:
        cursor = conexion.cursor()
        cursor.execute(
            """UPDATE servicio
            SET cliente_id=%s, tecnico_id=%s, descripcion=%s, estado=%s, fecha=%s
            WHERE id=%s""",
            (cliente_id, tecnico_id, descripcion.strip(), estado, fecha, servicio_id)
        )
        conexion.commit()
        if cursor.rowcount == 0:
            return jsonify({"success": False, "message": "Servicio no encontrado"}), 404
        return jsonify({"success": True, "message": "Servicio actualizado"}), 200
    except Exception as e:
```




```
        conexion.rollback()
        return jsonify({"success": False, "message": str(e)}), 500
    finally:
        if cursor: cursor.close()
        conexion.close()
```

14. DELETE – Eliminar Servicio

```
@servicios_bp.route("/api/servicios/<int:servicio_id>", methods=["DELETE"])
def eliminar_servicio(servicio_id):
    conexion = conectar()
    if conexion is None:
        return jsonify({"success": False, "message": "Sin conexión a BD"}), 500
    cursor = None
    try:
        cursor = conexion.cursor()
        cursor.execute("DELETE FROM servicio WHERE id=%s", (servicio_id,))
        conexion.commit()
        if cursor.rowcount == 0:
            return jsonify({"success": False, "message": "Servicio no encontrado"}), 404
        return jsonify({"success": True, "message": "Servicio eliminado"}), 200
    except Exception as e:
        conexion.rollback()
        return jsonify({"success": False, "message": str(e)}), 500
    finally:
        if cursor: cursor.close()
        conexion.close()
```

Elimina solamente registros de prueba. Una clave foránea o una regla de integridad puede impedir el DELETE.

15. Registrar Blueprint

```
backend/app/main.py

from app.routes.servicios import servicios_bp
app.register_blueprint(servicios_bp)
```

Guarda y reinicia Flask.

16. Probar catálogos con Thunder Client

1. GET http://127.0.0.1:5000/api/clientes/catalogo
2. GET http://127.0.0.1:5000/api/tecnicos/catalogo
3. GET http://127.0.0.1:5000/api/servicios

Los catálogos deben devolver id y nombre. Servicios debe devolver los datos y, mediante JOIN, cliente_nombre y tecnico_nombre.



17. Probar POST

```
POST http://127.0.0.1:5000/api/servicios

{
  "cliente_id": 1,
  "tecnico_id": 1,
  "descripcion": "Mantenimiento preventivo de equipo",
  "estado": "Pendiente",
  "fecha": "2026-10-07"
}
```

Usa IDs que realmente existan. Esperado: 201 Created.

18. Probar PUT y DELETE

```
PUT http://127.0.0.1:5000/api/servicios/ID

{
  "cliente_id": 1,
  "tecnico_id": 1,
  "descripcion": "Mantenimiento actualizado",
  "estado": "En proceso",
  "fecha": "2026-10-07"
}
```

Esperado: 200 OK. Después prueba DELETE sobre el mismo ID de prueba y vuelve a ejecutar GET.

19. Servicios React

En frontend/src/services/api.js agrega:

```
const API_SERVICIOS = "http://127.0.0.1:5000/api/servicios";

export async function obtenerServicios() {
  const r = await fetch(API_SERVICIOS);
  return { ok: r.ok, datos: await r.json() };
}

export async function obtenerClientesCatalogo() {
  const r = await fetch("http://127.0.0.1:5000/api/clientes/catalogo");
  return { ok: r.ok, datos: await r.json() };
}

export async function obtenerTecnicosCatalogo() {
  const r = await fetch("http://127.0.0.1:5000/api/tecnicos/catalogo");
  return { ok: r.ok, datos: await r.json() };
}

export async function crearServicio(servicio) {
  const r = await fetch(API_SERVICIOS, {
```




```
    method: "POST",
    headers: {"Content-Type":"application/json"},
    body: JSON.stringify(servicio)
  });
  return { ok: r.ok, datos: await r.json() };
}

export async function actualizarServicio(id, servicio) {
  const r = await fetch(`${API_SERVICIOS}/${id}`, {
    method: "PUT",
    headers: {"Content-Type":"application/json"},
    body: JSON.stringify(servicio)
  });
  return { ok: r.ok, datos: await r.json() };
}

export async function eliminarServicio(id) {
  const r = await fetch(`${API_SERVICIOS}/${id}`, {method:"DELETE"});
  return { ok: r.ok, datos: await r.json() };
}
```

20. Crear Servicios.jsx

Crea frontend/src/pages/Servicios.jsx. La interfaz debe cargar servicios, clientes y técnicos; usar SELECT para las relaciones y mostrar los nombres obtenidos por JOIN.

```
import { useEffect, useState } from "react";
import {
  obtenerServicios, obtenerClientesCatalogo, obtenerTecnicosCatalogo,
  crearServicio, actualizarServicio, eliminarServicio
} from "../services/api";

function Servicios() {
  const [servicios,setServicios]=useState([]);
  const [clientes,setClientes]=useState([]);
  const [tecnicos,setTecnicos]=useState([]);
  const [clienteId,setClienteId]=useState("");
  const [tecnicoId,setTecnicoId]=useState("");
  const [descripcion,setDescripcion]=useState("");
  const [estado,setEstado]=useState("Pendiente");
  const [fecha,setFecha]=useState("");
  const [editandoId,setEditandoId]=useState(null);
  const [mensaje,setMensaje]=useState("");

  const cargarDatos=async()=>{
    const [s,c,t]=await Promise.all([
      obtenerServicios(), obtenerClientesCatalogo(), obtenerTecnicosCatalogo()
    ]);
    if(s.ok) setServicios(s.datos.servicios || []);
```




```
  if(c.ok) setClientes(c.datos.clientes || []);
  if(t.ok) setTecnicos(t.datos.tecnicos || []);
};

useEffect(()=>{cargarDatos();},[]);

const limpiar=()=>{
  setClienteId(""); setTecnicoId(""); setDescripcion("");
  setEstado("Pendiente"); setFecha(""); setEditandoId(null);
};

const guardar=async(e)=>{
  e.preventDefault();
  if(!clienteId || !tecnicoId || !descripcion.trim()){
    setMensaje("Cliente, técnico y descripción son obligatorios"); return;
  }
  const datos={cliente_id:Number(clienteId),tecnico_id:Number(tecnicoId),
    descripcion,estado,fecha};
  const r=editandoId
    ? await actualizarServicio(editandoId,datos)
    : await crearServicio(datos);
  setMensaje(r.datos.message || "Operación realizada");
  if(r.ok){limpiar(); await cargarDatos();}
};

const editar=(s)=>{
  setEditandoId(s.id); setClienteId(s.cliente_id);
  setTecnicoId(s.tecnico_id); setDescripcion(s.descripcion);
  setEstado(s.estado || "Pendiente"); setFecha(s.fecha || "");
};

const eliminar=async(id)=>{
  if(!window.confirm("¿Eliminar este servicio de prueba?")) return;
  const r=await eliminarServicio(id);
  setMensaje(r.datos.message || "Operación realizada");
  if(r.ok) await cargarDatos();
};

return (
  <div className="dashboard">
    <h1>Servicios Técnicos</h1>
    <form onSubmit={guardar}>
      <select value={clienteId} onChange={e=>setClienteId(e.target.value)}>
        <option value="">Seleccione cliente</option>
        {clientes.map(c=><option key={c.id} value={c.id}>{c.nombre}</option>)}
      </select>

      <select value={tecnicoId} onChange={e=>setTecnicoId(e.target.value)}>
        <option value="">Seleccione técnico</option>
```




```
          {tecnicos.map(t=><option key={t.id} value={t.id}>{t.nombre}</option>)}
        </select>

        <input value={descripcion}
          onChange={e=>setDescripcion(e.target.value)}
          placeholder="Descripción"/>

        <select value={estado} onChange={e=>setEstado(e.target.value)}>
          <option>Pendiente</option><option>En proceso</option>
          <option>Terminado</option><option>Cancelado</option>
        </select>

        <input type="date" value={fecha}
          onChange={e=>setFecha(e.target.value)}/>

        <button type="submit">{editandoId ? "Actualizar" : "Registrar"}</button>
        {editandoId && <button type="button" onClick={limpiar}>Cancelar</button>}
      </form>

      {mensaje && <p>{mensaje}</p>}

      <table>
        <thead><tr>
          <th>ID</th><th>Cliente</th><th>Técnico</th>
          <th>Descripción</th><th>Estado</th><th>Fecha</th><th>Acciones</th>
        </tr></thead>
        <tbody>
          {servicios.map(s=><tr key={s.id}>
            <td>{s.id}</td><td>{s.cliente_nombre}</td><td>{s.tecnico_nombre}</td>
            <td>{s.descripcion}</td><td>{s.estado}</td><td>{s.fecha}</td>
            <td>
              <button onClick={()=>editar(s)}>Editar</button>
              <button onClick={()=>eliminar(s.id)}>Eliminar</button>
            </td>
          </tr>)}
        </tbody>
      </table>
    </div>
  );
}
export default Servicios;
```

Adapta los campos al resultado real de tu API.



21. Agregar la ruta protegida

```
import Servicios from "./pages/Servicios";

<Route path="/servicios"
  element={<RutaProtegida><Servicios /></RutaProtegida>}
/>
```

En Dashboard.jsx agrega:

```
<Link to="/servicios">Servicios</Link>
```

22. Prueba integral

4. Inicia MySQL, Flask y Vite.
5. Inicia sesión.
6. Confirma que existan clientes y técnicos.
7. Abre Servicios.
8. Comprueba que los SELECT carguen nombres.
9. Registra un servicio de prueba.
10. Verifica cliente y técnico por nombre.
11. Edita el servicio y cambia el estado.
12. Elimina el servicio de prueba.
13. Cierra sesión e intenta abrir /servicios directamente.

Resultado esperado: CRUD funcional, relaciones visibles y ruta protegida.

23. Errores frecuentes

Problema	Qué revisar
JOIN vacío	Claves foráneas y nombres de columnas.
500 SQL	SHOW CREATE TABLE y consulta.
Catálogo vacío	Debe haber clientes/técnicos.
400	Campos obligatorios y JSON.
Failed to fetch	Flask y CORS.
Nombres vacíos	Alias cliente_nombre/tecnico_nombre.
404 /servicios	App.jsx y RutaProtegida.

24. Evidencias (Captura de pantallas)

- SHOW CREATE TABLE de tablas relacionadas.
- servicios.py.
- Catálogos de clientes y técnicos.
- Thunder Client: GET, POST, PUT y DELETE.
- Formulario con SELECT de Cliente y Técnico.
- Servicio creado, editado y eliminado.



- Tabla con nombres obtenidos por JOIN.
- Ruta /servicios protegida.
- GitHub actualizado.

25. Registro de pruebas

Caso	Método/ruta	Esperado	Resultado	Evidencia
Catálogo clientes	GET /api/clientes/catalogo	200		
Catálogo técnicos	GET /api/tecnicos/catalogo	200		
Listar	GET /api/servicios	200		
Crear	POST /api/servicios	201		
Editar	PUT /api/servicios/{id}	200		
Eliminar	DELETE /api/servicios/{id}	200		
Validar	POST incompleto	400		

26. Preguntas de análisis

14. ¿Qué es una clave foránea?

Respuesta: _____

15. ¿Qué diferencia existe entre una consulta simple y una consulta con JOIN?

Respuesta: _____

16. ¿Por qué se necesitan catálogos para los SELECT?

Respuesta: _____

17. ¿Por qué es mejor mostrar el nombre que el ID al usuario?

Respuesta: _____

18. ¿Qué puede ocurrir al eliminar un cliente con servicios relacionados?

Respuesta: _____

19. ¿Qué ventaja tienen los endpoints de catálogo?

Respuesta: _____

20. ¿Cómo se relacionan React, Flask y MySQL?



Respuesta: _____

21. ¿Qué código podría reutilizarse en futuros módulos?

Respuesta: _____

27. Git y GitHub

```
git status
git add .
git commit -m "Agrega modulo de servicios y relaciones"
git push
```

Comprueba el commit en GitHub.